

Project Manager (AgentPM)

From Scratch to Advanced: Full-Stack Architecture, Autonomous AI Chatbot ("Jaydip Chatbot"), Tool Calling, Database Schemas, Reactive HTMX Frontend, and Core Terminology

Framework: Django 5.1 **AI Model:** Google Gemini (via OpenAI SDK) **Database:** SQLite3 via Django ORM

Frontend: Tailwind CSS + HTMX 1.9.10

1. Executive Overview & System Purpose

Project Manager (AgentPM) is a multi-tenant project and task execution platform. Traditional project trackers (like Jira or Trello) require manual clicks, complex form submissions, and cumbersome page reloads.

This project bridges traditional project dashboards with an **autonomous AI Copilot ("Jaydip Chatbot")**. Users can interact in two seamless ways:

- **Traditional Graphical Workspace:** Visual project boards with real-time completion progress meters, high-priority urgent focus queues, and quick inline task creator forms.
- **Natural Conversational AI Interface:** A conversational drawer where users issue commands in plain English (e.g. *"Plan my mobile app release with 4 milestone tasks"*, *"Assign the wireframe task to Sarah"*, or *"What tasks are due soon?"*).

When the AI executes an action, it modifies the backend SQLite database in real-time, which emits an event signal that automatically reloads the UI without a full page refresh.

2. Complete Technology Stack

Technology	Role in Stack	Technical Rationale & How It Operates
Python 3 & Django 5.1	Core Web Framework & Application Server	Implements the Model-View-Template (MVT) pattern. Handles user authentication, session security, database transactions, request routing, and partial HTML template rendering.
SQLite 3 (<code>db.sqlite3</code>)	Relational Database Engine	Zero-configuration, serverless, file-based relational database. Managed strictly via Django's Object-Relational Mapper (ORM), guaranteeing ACID transactional integrity.
Google Gemini API <code>gemini-3.6-flash</code>	Large Language Model (LLM) Engine	Provides multi-turn reasoning and autonomous function calling. Accessed through the OpenAI-compatible REST endpoint (<code>generativelanguage.googleapis.com/v1beta/openai/</code>) with fallback support.
HTMX 1.9.10	Reactive AJAX Frontend Engine	Enables seamless DOM swapping, event-driven re-rendering (via <code>HX-Trigger</code> headers), and live typing states without bulky SPA frameworks like React or Angular.
Tailwind CSS	User Interface Styling	Utility-first CSS delivering a modern dark-mode aesthetic (slate-950 backdrop, cyan/sky accents, glassmorphic blurred panels, and responsive grid layouts).
Marked.js	Markdown Rendering	Parses raw markdown from AI assistant responses (bullet lists, bold highlights, code formatting) into safe, styled HTML inside the chat bubbles.

3. Database Architecture & Schema Deep-Dive

The database is structured around four primary models defined in `projects/models.py`. Multi-tenancy is enforced at the database level: every entity links directly or indirectly to the authenticated Django `User`.

Entity Relationship Diagram (ERD)



Model Attributes & Business Logic

Model	Field / Method	Purpose & Technical Implementation
Project	<code>user</code> (ForeignKey)	Points to Django's <code>User</code> . If the user account is deleted, all their projects are erased via <code>CASCADE</code> .
	<code>completion_percentage</code>	Dynamic property: computes <code>(completed_tasks / total_tasks) * 100</code> . Used directly by templates to draw progress bars.
	<code>Meta.indexes</code>	B-Tree composite indexes on <code>['user', 'status']</code> and <code>['user', '-created_at']</code> for ultra-fast query execution.
Task	<code>priority</code>	Enum: <code>LOW</code> , <code>MEDIUM</code> , <code>HIGH</code> . High-priority unfinished tasks are filtered into the top urgent queue.
	<code>assigned_to</code>	Stores the team member handle (e.g. <code>@sarah</code>). Can be set manually or via natural language AI tools.
	<code>Meta.ordering</code>	<code>['is_completed', '-priority', 'due_date']</code> ensures pending, high-priority, soon-due tasks float to the top.
ChatMessage	<code>role</code>	Enum choices: <code>user</code> , <code>assistant</code> , <code>system</code> , <code>tool</code> representing the OpenAI conversation schema.
	<code>tool_calls</code> (JSON)	Preserves raw function call payloads returned by the AI for inspection and audit trails.

4. The AI Autonomous Agent ("Jaydip Chatbot")

The AI system in `projects/ai_agent.py` is an implementation of the **ReAct (Reasoning + Acting) Agent Pattern** using function calling.

Step 1: OpenAI SDK Client with Gemini Gateway

```
client = OpenAI(  
    api_key=settings.GEMINI_API_KEY,  
    base_url='https://generativelanguage.googleapis.com/v1beta/openai/',  
)
```

By pointing the official OpenAI client library to Google's v1beta OpenAI-compatible endpoint, standard chat completions and function calls work seamlessly with Gemini models.

Step 2: Resilient Quota Fallback Architecture (`\_call\_gemini`)

To prevent user-facing interruptions caused by free-tier rate limits (HTTP 429), the agent employs an automated fallback ladder:

Model Fallback Ladder:

`gemini-3.6-flash` → `gemini-3.5-flash` → `gemini-flash-latest` → `gemini-3.1-flash-lite`

Step 3: Tool Definitions & Function Schemas

The model is provided with 8 structured JSON tool definitions (`TOOL_SCHEMAS`) describing what actions are available:

Tool Name	Input Parameters	What It Does in the Database
<code>list_projects</code>	None	Queries all projects and tasks for the user and serializes them to JSON for the model's reasoning.
<code>create_project</code>	<code>title</code> , <code>description</code>	Inserts a new <code>Project</code> record belonging to <code>request.user</code> .
<code>create_task</code>	<code>title</code> , <code>project_title</code> , <code>priority</code> , <code>due_date</code> , <code>assigned_to</code>	Resolves the target project, assigns priority, and creates the <code>Task</code> record. Auto-creates "General Tasks" project if needed.
<code>assign_task</code>	<code>task_identifier</code> , <code>project_identifier</code> , <code>assigned_to</code>	Updates task assignee or transfers task between different projects.
<code>update_task_status</code>	<code>task_identifier</code> , <code>is_completed</code>	Marks a task completed or pending by name or numeric ID.
<code>delete_task</code>	<code>task_identifier</code>	Removes the matching task from the database.
<code>delete_project</code>	<code>title</code> or <code>project_id</code>	Deletes the project and cascades deletion to all contained tasks.

Step 4: Flexible Entity Resolution Helpers

Users often refer to tasks colloquially (e.g. "Delete wireframe" instead of "Delete task ID 12"). Helper functions resolve targets intelligently:

- Checks if input is a numeric ID (e.g. `"5"` → ID match).
- Performs exact case-insensitive match (`title__iexact=query`).
- Performs substring match (`title__icontains=query`).
- Falls back to the user's sole project if only one exists.

Step 5: The Multi-Turn ReAct Agent Execution Loop

1. **Record User Turn:** Creates a `ChatMessage(role='user')` in SQLite.
2. **Assemble History:** Fetches up to 8 recent user/assistant messages to preserve conversational context while minimizing token overhead.
3. **Inject System Directive:** Injects date, role persona ("Intelligent Project Manager"), markdown formatting instructions, and tool constraints.
4. **Inference Loop (Max 5 Turns):**
 - Calls Gemini API with payload and tools.
 - If the response contains **tool_calls**:
 - Iterates each requested tool.
 - Executes matching Python function in `AVAILABLE_TOOLS` scoped to `user`.
 - Sets `did_mutate = True` if data was changed.
 - Appends tool result to the conversation with `role: 'tool'`.
 - Loops back to Gemini with the tool outputs.
 - If the response contains text without tool calls:
 - Saves final assistant reply in `ChatMessage`.
 - Returns `(final_reply, did_mutate)` to the Django view.

5. Reactive Frontend & HTMX Event Synchronization

Traditional applications either reload the entire browser page or require complex frontend state management (Redux, Vuex). AgentPM achieves full reactivity using **Django + HTMX**:

The `HX-Trigger: projectStateChanged` Pattern

1. Chat View Emits Event: When the AI performs any mutation, `views.send_chat` attaches an HTTP response header:

```
if did_mutate:
    response['HX-Trigger'] = 'projectStateChanged'
```

2. Browser Listens & Auto-Updates: The HTML containers listen for this custom event on the document body:

```
<!-- Project Board Container in templates/dashboard.html -->
<div id="project-board"
    hx-get="{% url 'project_board_partial' %}"
    hx-trigger="projectStateChanged from:body">
    {% include "partials/project_board.html" %}
</div>
```

3. Instant Visual Sync: When Gemini creates a task in the background, HTMX silently fires an AJAX GET request, Django returns the re-rendered `project_board.html` fragment, and HTMX replaces the inner HTML instantly. The user sees their new project cards and progress bars appear without touching refresh!

Chatbot Drawer Windowing Modes

The AI drawer (`#ai-drawer`) includes three user-selectable window states implemented via lightweight vanilla JavaScript:

- **Default Compact View (384px width):** Anchored on the right side of the workspace, allowing simultaneous board view and chatting.
- **Maximized / Expanded View (820px width):** Expands across the screen for wide-angle roadmapping, long checklists, and detailed project breakdowns.
- **Minimized Floating Mode:** Completely folds away into a bottom-right floating pill button with an animated emerald pulse beacon, freeing 100% of workspace canvas.

6. Project Directory & File Guide

File Path	Description & Responsibilities
core/settings.py	Application configuration: loads environment variables (<code>.env</code>), configures SQLite database, installed apps, middleware, and Gemini API credentials.
core/urls.py	Top-level URL routing table. Mounts Django admin at <code>/admin/</code> and includes <code>projects.urls</code> at the root <code>/</code> .
projects/models.py	Defines <code>Project</code> , <code>Task</code> , and <code>ChatMessage</code> ORM tables, relationships, cascade behaviors, and indexes.
projects/ai_agent.py	The autonomous bot engine. Contains OpenAI-Gemini client, fallback handler, tool functions, schemas, and ReAct loop.
projects/views.py	Controllers for user registration, workspace dashboard, HTMX action handlers (create/toggle/delete tasks), and <code>send_chat</code> .
projects/urls.py	URL routing for dashboard, login/logout, task toggles, and chat API endpoints.
templates/base.html	Master HTML5 layout. Loads Tailwind CDN, HTMX CDN, Marked.js, and configures global CSRF headers on HTMX requests.
templates/dashboard.html	Main workspace interface: top navigation, urgent tasks, project boards, add task/project modals, and the AI chatbot drawer.
templates/partials/*.html	Modular HTML fragments (<code>project_board.html</code> , <code>task_item.html</code> , <code>urgent_tasks.html</code> , <code>chat_stream.html</code>) swapped by HTMX.

7. Glossary of Key Terms

Term	Definition & Context in this Project
ORM (Object-Relational Mapping)	A programming technique that maps database tables to Python classes. Allows developers to interact with the database using Python objects (<code>Task.objects.create(...)</code>) rather than writing raw SQL.
Multi-Tenancy	An architecture where multiple users share the same system and database, but their records are strictly isolated. In AgentPM, every database operation verifies <code>user=request.user</code> .
Foreign Key & CASCADE	A relational link between tables. <code>CASCADE</code> ensures that if a parent record is deleted (e.g. a Project), all child records (Tasks) are automatically purged to prevent orphan data.
Function / Tool Calling	A capability where the LLM does not just reply with text, but emits a structured JSON object specifying a function name and arguments to execute programmatic changes.
ReAct Pattern	"Reasoning + Acting". A design pattern where an AI alternates between thinking, executing external tools, observing the output, and arriving at a conclusive user response.
HTMX Partial Swapping	The technique of returning small chunks of server-rendered HTML from the backend and inserting them directly into specified DOM elements without redrawing the whole page.
CSRF Protection	Cross-Site Request Forgery mitigation. Django injects a cryptographically signed token into forms, verified on every state-mutating POST request.
Database Indexing	Data structures created on specific database columns (e.g. <code>user</code> , <code>status</code>) allowing the database engine to locate records in $O(\log n)$ time rather than full table scans.

Architecture Complete

This document serves as the exhaustive reference manual for the AgentPM system architecture, database schema, AI engine, and interactive frontend.